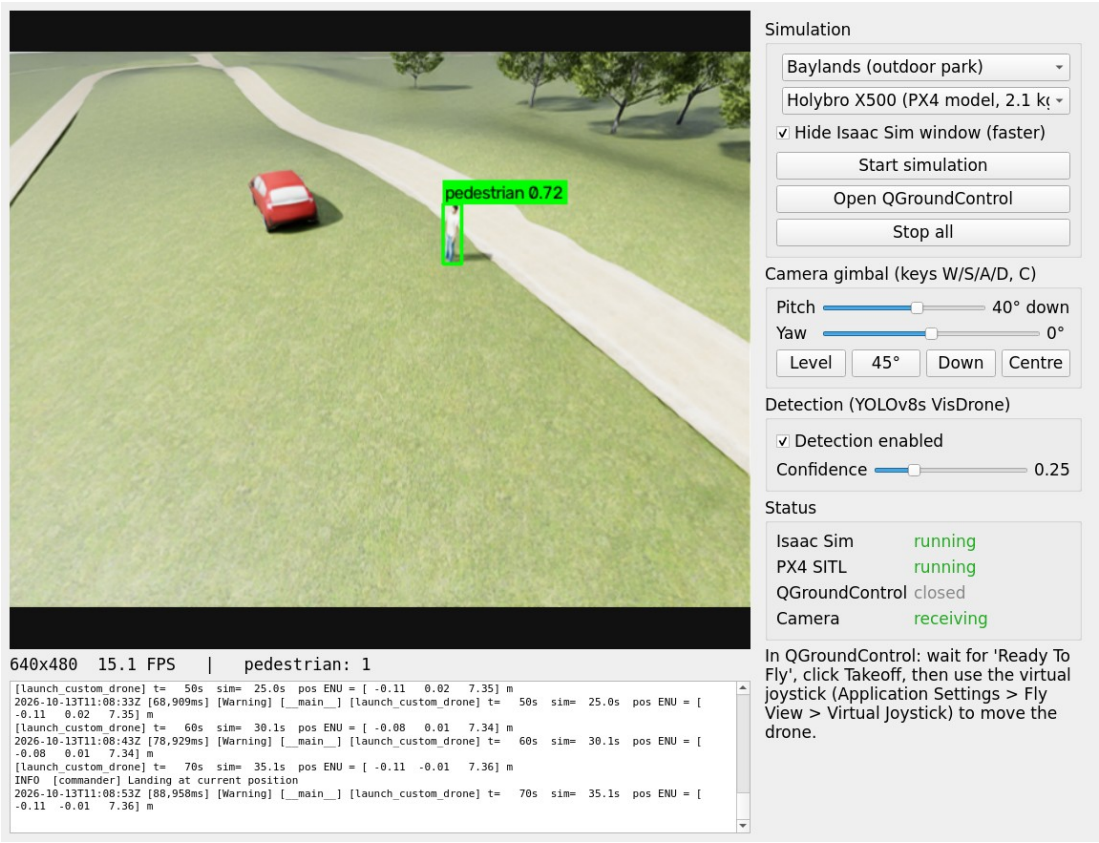
Drone Simulation and Onboard Detection Pipeline

Isaac Sim 6.0 · Pegasus Simulator · PX4 SITL · QGroundControl · ROS 2 Humble · YOLOv8s (VisDrone)

Technical report: what was built, how it was built, how it works, and where it can go next



Amanullah Naseer

Department of Computer Engineering

National University of Sciences and Technology (NUST)

13 October 2026

Contents

1. Summary
2. Goals and starting point
3. Workspace clean-up
4. System architecture
5. Components and tools
6. Drone models
7. Gimbal camera
8. How it was built
9. How it works at run time
10. Using the application
11. Testing and results
12. Limitations and known issues
13. Improvements
14. Future applications
15. Research directions
Appendix A. Files and folders
Appendix B. Commands
Appendix C. Asset sources and licences

1. Summary

This project turns a PC that already had NVIDIA Isaac Sim, the Pegasus Simulator and PX4-Autopilot installed into a one-click drone simulation and perception testbed. A PyQt5 desktop application lets the user pick one of four drone models, starts Isaac Sim with that drone flying on the real PX4 autopilot in an outdoor park (Baylands) containing a car and a person, opens QGroundControl for take-off and joystick flight, and shows the drone's gimbal camera with live YOLO object detection drawn on the video. The camera gimbal can be pitched and yawed from the application during flight.

The home folder was also reorganised: about 35 GB not needed for this work (including a duplicate 25 GB Isaac Sim install) was moved, not deleted, into “Not needed Now” with a manifest of every original path.

Item	Result
Launcher	app.py + start_app.sh, also in the desktop applications menu
Drone models	Pegasus quad (1.5 kg), Holybro X500 from PX4's own model (2.1 kg), 5-inch FPV racer (0.6 kg), 3DR Iris
Scene	Baylands park (Gazebo model, ~870k triangles) + red hatchback + standing person
Flight	PX4 v1.14.3 SITL in lockstep at 250 Hz; X500, FPV and Pegasus each armed, took off to ~7.2 m, hovered and landed in automated tests
Gimbal	2-axis (pitch -30° to 90°, yaw ±170°), 60°/s slew, sliders, presets and W/S/A/D keys in the app
Detection	YOLOv8s fine-tuned on VisDrone on the GPU: person detected consistently (0.54–0.77), car only occasionally (~0.37)
Clean-up	15 items moved, manifest in ~/Not needed Now/MANIFEST.md

2. Goals and starting point

The request: an application that launches Isaac Sim and QGroundControl, spawns a PX4 multirotor in Isaac Sim together with a car, a person and an outdoor scene (Baylands or mountains), lets the user take off and fly with the QGC joystick, and shows the camera feed with detection in the application. The home folder should keep only what this task needs. A follow-up asked for a drone model selector (adding the Holybro X500 from PX4's Gazebo files and an FPV racer) and for moving the gimbal camera from the application during flight.

What already existed on the PC (checked, not assumed):

- Isaac Sim 6.0.0 (rc.59) in ~/isaacsim, plus an identical copy in ~/isaac-sim-standalone-6a.0.0-linux-x86_64.
- Pegasus Simulator 6.0.0 in ~/PegasusSimulator, installed in editable mode into Isaac Sim's Python.
- PX4-Autopilot v1.14.3 in ~/PX4-Autopilot, built for SITL, with Gazebo (gz) models for x500, x500_depth, x500_vision, px4vision, standard_vtol and rc_cessna. There is no racer airframe or racer model in this PX4 version, and the Gazebo-Classic model folder (e.g. typhoon_h480 hexacopter meshes) is not checked out.
- A verified pipeline in ~/drone_px4_isaac: launch_custom_drone.py spawns a Pegasus drone with the PX4 MAVLink backend, attaches a camera and publishes it over ROS 2.
- A Python 3.10 environment with PyTorch 2.5.1 + CUDA, Ultralytics 8.4 and PyQt5 (ml_project/myenv).
- Detector weights best8s.pt: YOLOv8s fine-tuned on VisDrone (pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, motor); used as models/best.pt.

- ROS 2 Humble in /opt/ros/humble; GPU RTX 4060 8 GB.

3. Workspace clean-up

Rules: move, never delete; leave dotfiles, configuration and the standard desktop folders alone; nothing outside the home folder. The keep/move list was shared before anything moved.

Kept in home	Why
isaacsim	The Isaac Sim install used (ISAACSIM_PATH in .bashrc)
PegasusSimulator	Editable-installed into Isaac Sim's Python; moving it breaks imports
PX4-Autopilot	Built SITL binary and airframes that Pegasus launches
drone_px4_isaac	The pipeline: app, simulation script, assets, model, Python environment
Desktop, Documents, Downloads, ...	Standard folders and personal files, untouched

Moved to ~/Not needed Now	Note
isaac-sim-standalone-6a.0.0-linux-x86_64	25 GB byte-identical duplicate of Isaac Sim
isaac sim	Older PX4 tree (v1.18 beta), second Pegasus checkout, old scripts
ml_project	Tracking/guidance experiments, other weights (venv and best8s.pt relocated first)
fanet_semantic	Earlier FANET testbed (aerial scene, ground-truth logger)
DroneTrackingLatest..., FPV Tracking...	Older projects
Loose scripts in home	my_sim.py, launcher.py, kamikaze_gcs.py, swarm.launch.py, ...

Every move is recorded in ~/Not needed Now/MANIFEST.md; restoring is one mv back to the listed path.

4. System architecture

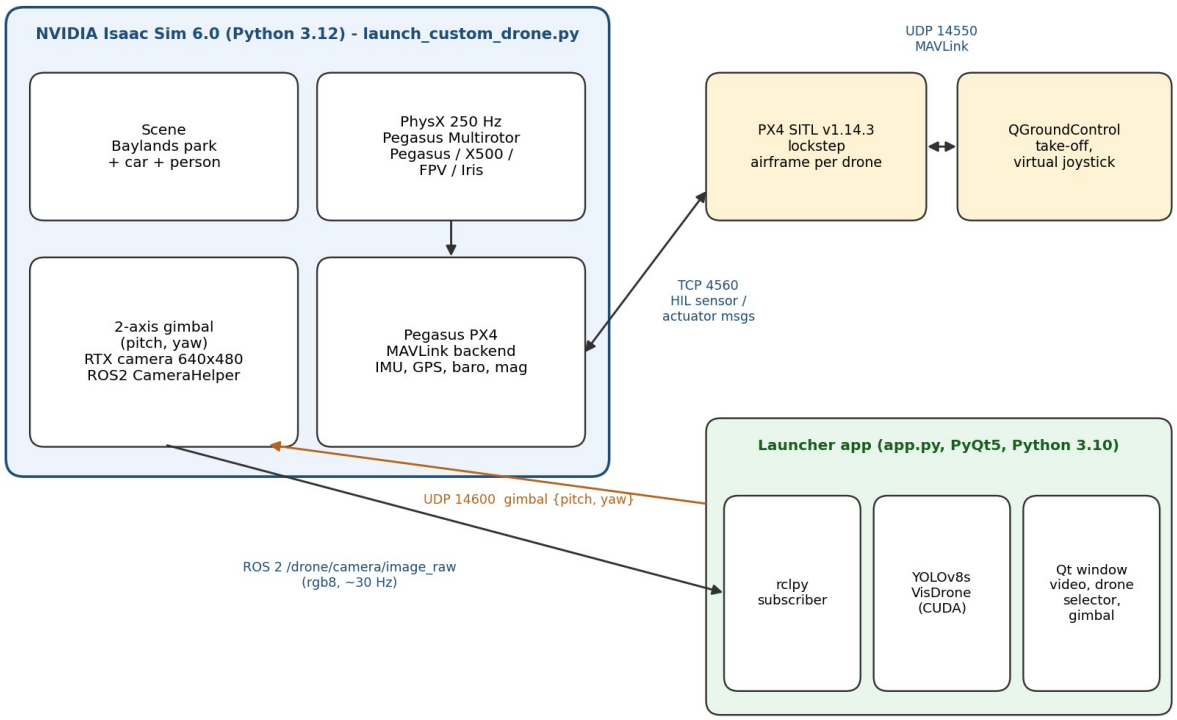


Figure 1. Processes and data flow. Blue: Isaac Sim; yellow: autopilot and ground station; green: launcher application; brown: gimbal commands.

Four processes cooperate. Isaac Sim owns the world: physics, rendering, the drone's rigid bodies, the gimbal and the camera. PX4 is the real flight-control software compiled for Linux; it sees only simulated sensor messages, exactly as it would see real sensors through drivers. QGroundControl talks to PX4 over MAVLink as it would to a real vehicle. The launcher application reads images over ROS 2 and sends gimbal angles over a local UDP socket; it never commands the flight, so take-off and flying use the same path a pilot uses in the field.

5. Components and tools

Component	Version	Role in this pipeline
NVIDIA Isaac Sim	6.0.0-rc.59	PhysX at 250 Hz, RTX rendering, USD scene, OmniGraph, bundled ROS 2 bridge
Pegasus Simulator	6.0.0	Multirotor model, thrust curve, IMU/GPS/baro/mag models, PX4 MAVLink backend
PX4-Autopilot	v1.14.3	Flight controller in SITL, lockstep; airframes isaac_iris, isaac_x500, isaac_fpv
QGroundControl	AppImage	Arming, take-off, landing, virtual joystick
ROS 2 Humble	Isaac internal + /opt/ros	Camera transport (sensor_msgs/Image), /clock
Ultralytics YOLOv8s	8.4.118	Detector fine-tuned on VisDrone (best.pt)
PyTorch	2.5.1 + CUDA 12.1	GPU inference on the RTX 4060
PyQt5 / OpenCV	5.15 / 5.0	Application window, box drawing, image conversion
Gazebo Fuel / PX4 gz models	OpenRobotics / PX4	Baylands, hatchback, standing person; X500 meshes
usd-core, trimesh, pycollada	pip	Offline conversion of DAE/OBJ/STL meshes to USD

Why these choices

- **Isaac Sim + Pegasus instead of Gazebo:** RTX rendering gives camera images much closer to real footage, which matters when evaluating a vision model; Pegasus is a maintained PX4 bridge for Isaac Sim.
- **PX4 SITL in lockstep:** PX4 waits for each sensor message, so the autopilot stays correct even when rendering makes the simulation slower than real time.
- **ROS 2 for the camera:** the same subscriber works with a real camera driver, and any ROS 2 node (recorder, tracker, network emulator) can tap the topic.
- **Gazebo models:** the Isaac 6.0 asset library has people but no road vehicles and no Baylands; the Gazebo versions are the Baylands world PX4 users know and are openly licensed.
- **Two Python environments:** Isaac Sim needs Python 3.12; the app uses Python 3.10 so it can load the system ROS 2 Humble rclpy together with PyTorch.

6. Drone models

The app's Drone selector offers four models. Pegasus needs a specific USD layout: a body rigid body with mass and inertia, four rotor rigid bodies named rotor0 to rotor3 in PX4 quad-X order (front-right, rear-left, front-left, rear-right) joined to the body by revolute joints, simple convex colliders, Z-up and metres. Each model also needs a PX4 airframe whose rotor geometry matches the USD.

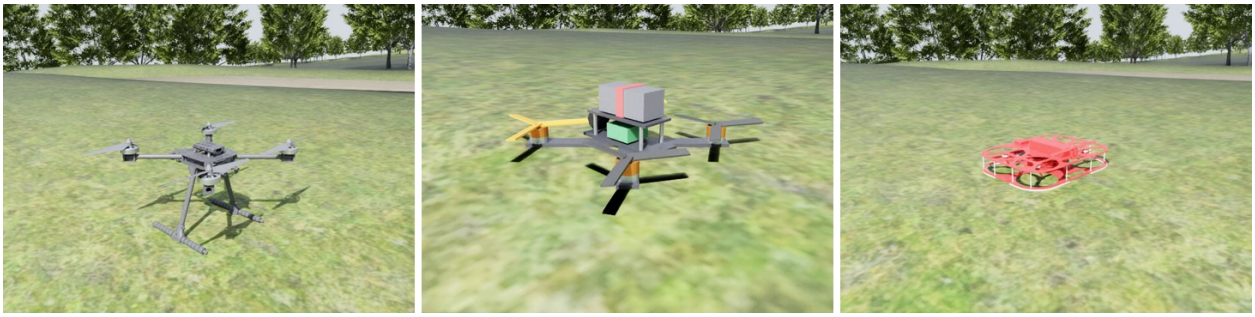


Figure 2. Holybro X500, FPV 5-inch racer and the Pegasus quad in the Baylands scene (rendered in Isaac Sim).

Model	Mass	Size	Thrust/ weight	PX4 airframe	Source
Pegasus quad	1.50 kg	~0.45 m	2.8	isaac_iris (22000)	existing pegasus.usd
Holybro X500	2.06 kg	0.49 m diag.	2.0	isaac_x500 (22001)	PX4 gz model x500 meshes, mass, inertia
FPV 5-inch racer	0.62 kg	0.22 m diag.	4.5	isaac_fpv (22002)	built from primitives
3DR Iris	1.50 kg	0.55 m	2.8	isaac_iris (22000)	Pegasus stock

6.1 Holybro X500 from PX4's Gazebo model

The X500 is built from the files PX4 already ships in ~/PX4-Autopilot/Tools/simulation/gz/models/x500. Rather than a generic SDF import followed by manual restructuring in the GUI, a script (tools/build_drones.py) reads the same carbon-frame, motor-base, motor-bell and propeller meshes at the poses given in model.sdf, uses the SDF's mass (2.0 kg + 4 × 16 g rotors) and inertia, copies the five box colliders (frame plate, two legs, two skids), and writes the Pegasus layout directly. The process is reproducible and needs no GUI steps.

A matching PX4 airframe, 22001_isaac_x500, combines the X500 rotor geometry from PX4's 4001_gz_x500 (±0.174 m) with the MAVLink HIL simulator interface Pegasus uses. The rotor constant is the Gazebo value (8.54858e-6 N/(rad/s)²) with a 1100 rad/s limit, giving a thrust-to-weight of about 2.

6.2 FPV 5-inch racer

PX4 v1.14.3 on this PC has no racer airframe and no racer model files, and downloading third-party meshes was out of scope, so the FPV quad is built from primitives with real 5-inch proportions: 220 mm motor-to-motor X-frame, stacked flight controller, top-mounted battery with strap, FPV camera, 2207-size motors and tri-blade props (orange at the front, as pilots use for orientation). Mass 0.62 kg, inertia (1.2, 1.3, 2.2) × 10^{-3} kg·m².

Its airframe 22002_isaac_fpv uses the ±0.078 m rotor geometry and scales the rate-controller gains down (roll/pitch P 0.03, I 0.04, D 0.0006; yaw P 0.04) because the small frame has about six times more angular authority than the Iris; with the default gains it would oscillate. Rotor constant 5.6e-6 gives a thrust-to-weight of about 4.5, and maximum rates are raised to 360°/s.

6.3 Hexacopter

A hexacopter was part of the original request. PX4 includes a hexacopter airframe (6011 typhoon_h480), but its Gazebo-Classic meshes are not on this PC and the four Pegasus models above are quad-X. Adding one needs a six-rotor USD, a six-rotor thrust curve in the simulation script and a hex airframe; see Improvements.

7. Gimbal camera

The camera is mounted on a two-axis gimbal attached to the drone body: yaw about the body Z axis, then pitch about the gimbal Y axis (positive pitch looks down). The simulation script listens on UDP 127.0.0.1:14600 for JSON messages {"pitch": degrees, "yaw": degrees} and moves the gimbal towards the newest command at 60°/s, like a servo, within pitch −30° to 90° and yaw ±170°. A small housing makes the gimbal visible from outside.

In the app the Camera gimbal panel has pitch and yaw sliders, presets (Level, 45° down, Straight down, Centre) and keyboard shortcuts W/S (pitch) and A/D (yaw), C (centre). The current angles are re-sent every second, so a freshly started simulation picks them up. Each model mounts the gimbal at a sensible place: under the nose for the X500 (0.22 m forward, clear of the landing gear), at the front of the FPV frame, and at the front of the Pegasus quad.

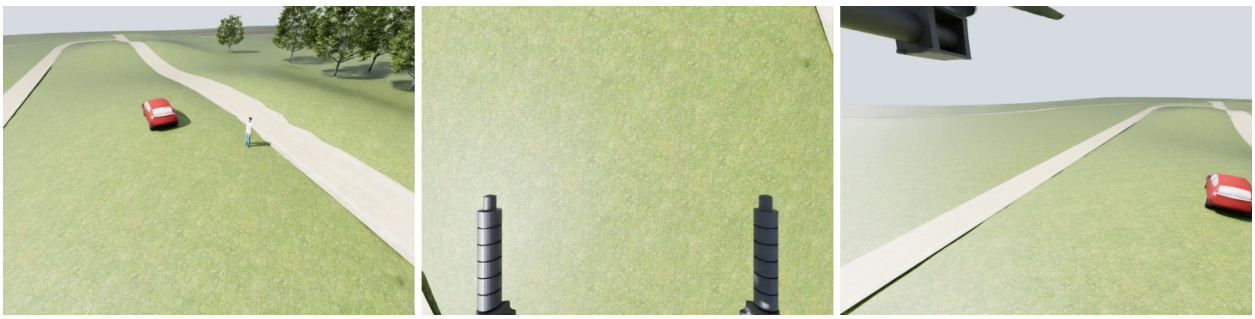


Figure 3. X500 in flight at 7.2 m, gimbal commanded from the app interface: 35° down, straight down (landing gear visible), and 20° down with 60° yaw left.

8. How it was built

8.1 Survey

Every top-level folder was measured; the active Isaac Sim was identified from .bashrc, the Pegasus install from Isaac Sim's site-packages, the PX4 path from Pegasus' configs.yaml, and the detector by loading each weight file and reading its classes.

8.2 Scene assets

Baylands, the hatchback and the standing person were downloaded from Gazebo Fuel. tools/mesh_to_usd.py loads the Collada/OBJ meshes with trimesh, applies the Gazebo pose, and writes a Z-up, metre-scale USD with UsdPreviewSurface materials; transparent textures (leaves, markings) get opacity cut-outs, duplicate textures are merged. tools/add_colliders.py marks only ground surfaces (grass, roads, paths, sand) as static mesh colliders, so physics start-up stays fast. Terrain heights sampled from the grass mesh put the car and person on the ground of the gentle hill in front of the drone.

8.3 Simulation script

launch_custom_drone.py was extended, keeping its verified flight behaviour: --env baylands, --objects, the gimbal (GIMBAL_PATH, --gimbal-port, --cam-yaw) and --rolling-moment for per-model yaw drag. A gimbal port that is already taken only disables gimbal control instead of stopping the simulation.

8.4 Launcher application

- **Drone presets:** each entry maps to the USD, PX4 airframe, thrust constants and gimbal mount offset.
- **Latest-frame-only detection:** a ROS 2 thread keeps only the newest image, so the display never lags behind even if inference is slower than the camera.
- **Clean shutdown:** the simulator runs in its own process group (setsid); Stop sends SIGTERM to the group so Isaac Sim's Python behind the python.sh wrapper stops PX4 cleanly. This bug was found and fixed during testing.

Simulated battery: PX4's SITL battery drains to 50 % in 60 s by default. On a long X500 flight in the app this produced a “Battery unhealthy” failsafe after take-off. The three Isaac airframes (22000_isaac_iris, 22001_isaac_x500, 22002_isaac_fpv) therefore set SIM_BAT_MIN_PCT = 100, and the retest ran without failsafes.

start_app.sh sources ROS 2 Humble and runs the app with the project's virtual environment; run_sim.sh removes /opt/ros from the environment it hands to Isaac Sim, which must use its own Python 3.12 ROS 2 libraries.

9. How it works at run time



Figure 4. Start-up sequence (wall-clock times from the test runs).

- Each 4 ms physics step, Pegasus reads the drone state from PhysX, generates noisy IMU, GPS, barometer and magnetometer readings, and sends them to PX4 as HIL_SENSOR / HIL_GPS messages over TCP 4560.
- PX4 runs EKF2 and its controllers and returns motor commands (HIL_ACTUATOR_CONTROLS); Pegasus converts them to rotor speeds with a quadratic thrust curve and applies forces and yaw moments to the rotor bodies.
- Lockstep advances simulated time only when PX4 has answered; measured real-time factor is about 0.5 in Baylands with the Isaac Sim window hidden.
- Rendering runs at 60 Hz simulated time; the gimbal camera publishes every second frame (30 Hz) on /drone/camera/image_raw with camera_info and /clock.
- PX4 sends telemetry to QGroundControl on UDP 14550; the pilot arms, takes off and flies with the virtual joystick (MANUAL_CONTROL messages), exactly as with a real PX4 drone.
- The app runs YOLOv8s on each newest frame and draws colour-coded boxes (people green, car orange).

10. Using the application

```
~/drone_px4_isaac/start_app.sh      (or “Drone Sim + Detection” in the applications menu)
```

- Choose a Scene and a Drone, then click Start simulation (Isaac Sim needs about a minute; tick “Hide Isaac Sim window” for more speed).
- Click Open QGroundControl, wait for “Ready To Fly”, press Takeoff.
- Enable the virtual joystick in QGC (Application Settings > General > Fly View) and fly towards the car and the person.
- Point the camera with the gimbal sliders, presets or W/S/A/D keys; detections appear in the video.
- Adjust the detection confidence (default 0.25) or switch detection off; Stop all ends everything.

11. Testing and results

All tests ran automatically on this PC with the Isaac Sim window hidden: arm and take off to 8 m over MAVLink (UDP 14540, the same commands QGC sends), hover, move the gimbal through three positions while grabbing camera frames, land, then stop the simulation.

Model	Take-off	Hover / landing	Real-time factor	Camera (wall clock)	Gimbal
Pegasus quad	7.21 m	90 s hover, landed	≈ 0.47	8.5–13 Hz	n/a (before gimbal)
Holybro X500	7.20 m	120 s hover, landed	≈ 0.52	12–13 Hz	35°, 90°, 20°/yaw 60° OK
FPV 5-inch	7.21 m	120 s hover, landed	≈ 0.51	12–14.5 Hz	35°, 90°, 20°/yaw 60° OK

Position hold was steady for all three (for example the FPV stayed within 0.13 m of the take-off point at 7.15–7.17 m). The application itself was driven offscreen with the X500 selected: simulation start, live feed at 13–15 FPS with the person detected in 42 one-second samples, Stop all with no leftover processes.

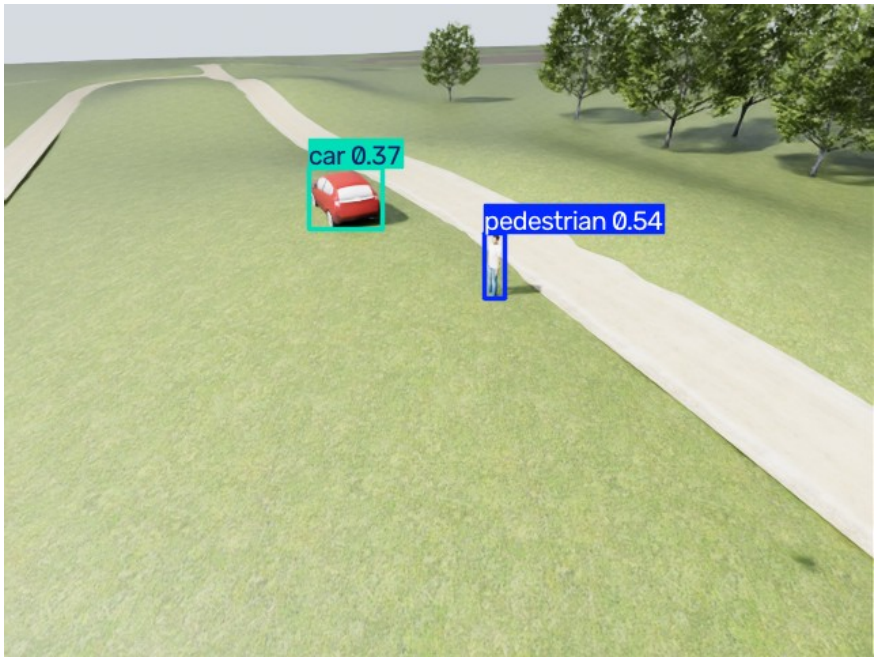


Figure 5. YOLOv8s (VisDrone) on the Pegasus quad's camera at 7.2 m: pedestrian 0.54, car 0.37.

Detection metric	Value
Person	detected consistently, confidence 0.54–0.77
Car	intermittent: confidence ≈ 0.37; seen in 6 sampled seconds of the Pegasus app run, 0–1 in X500 runs
App camera rate	13–14 FPS wall clock (30 Hz simulated)

12. Limitations and known issues

- **Quadcopters only.** No hexacopter yet (see 6.3).
- **FPV model is not a racer airframe from PX4.** PX4 v1.14.3 has none; the frame is modelled from primitives and tuned for stable hover, not for acro flips.
- **Slower than real time.** Lockstep keeps flight correct but the world runs at about half speed.

- **Static targets.** The car and the person do not move.
- **Car detection.** VisDrone is mostly steep aerial imagery; car confidence is low at shallow angles. Pitching the gimbal down from higher altitude helps.
- **Missing drone materials.** pegasus.usd references MDL materials that are not on disk, so Isaac Sim logs material errors at start; cosmetic only.
- **Gimbal not linked to QGC.** The gimbal is driven from the app, not from QGC's gimbal controls.
- **Interactive check pending.** Tests were headless; the visible Isaac Sim and QGC windows use the same code paths but were not exercised by hand.

13. Improvements

Area	Improvement
Vehicles	Hexacopter (6-rotor USD + PX4 hex airframe); FPV acro tuning; real FPV mesh if one is provided
Gimbal	Expose it as a MAVLink gimbal (Gimbal Protocol v2) so QGC can drive it; stabilise in world frame
Targets	Animate the person (walk cycles) and drive the car along the park road; add crowds and traffic
Detection	Fine-tune on synthetic frames from this scene (Isaac Replicator gives free labels); TensorRT FP16
Tracking	ByteTrack/BoT-SORT IDs, then closed-loop following via MAVLink offboard on UDP 14540
Speed	Lower viewport resolution, render only the camera when headless, cache Baylands as USDC
Ground truth	Publish true object poses and projected boxes to compute precision/recall automatically
Robustness	Weather and lighting randomisation (sun angle, fog, dusk)
Usability	Remember settings, record video / rosbag, one-click screenshots

14. Future applications

- **Search and rescue:** rehearse search patterns and measure how altitude, gimbal angle and speed change the probability of spotting a person.
- **Traffic and parking monitoring:** VisDrone classes map directly to vehicle counting and flow estimation.
- **Inspection and patrol:** repeatable missions with detection alerts.
- **Pilot and operator training:** safe practice of QGC procedures and gimbal handling with realistic video.
- **Software-in-the-loop testing:** regression-test autopilot parameters or detector updates before real flights; compare airframes (X500 vs FPV) in identical scenes.
- **Hardware-in-the-loop:** replace SITL with a real flight controller and run the same ROS 2 subscriber on an onboard computer such as a Jetson.

15. Research directions

Link to the FANET semantic-communication work

The earlier FANET testbed on this PC studied UAVs that send compact semantic messages, a few tens of bytes describing what was seen, instead of raw video over a flying ad-hoc network. This pipeline is a natural front end for that study:

- Detector output (class, confidence, box, time, drone pose, gimbal angles) is exactly the content of a semantic message: 44–72 bytes per frame versus about 0.9 MB for a raw 640x480 RGB frame, more than four orders of magnitude less.

- PX4's GPS pose plus the known gimbal angles let boxes be geo-referenced to ground coordinates, so messages from different UAVs can be fused at the receiver.
- Several Pegasus vehicles (vehicle_id 0, 1, 2, ...), possibly of different types (X500 survey drone + FPV interceptor), can share one scene with their own PX4 instances to emulate a heterogeneous FANET; link delay and loss can be injected with network namespaces or ns-3.
- Questions: task accuracy versus bandwidth, age of information of detections, effect of packet loss on cooperative tracking, and gimbal pointing chosen to maximise information sent per byte.

Other directions

- Sim-to-real transfer of aerial detectors; domain randomisation and style transfer.
- Active perception: choosing altitude and gimbal angle to maximise detection confidence (RL or information-gain planning in the loop with PX4).
- Platform comparison: how frame size, agility (FPV vs X500) and gimbal stability change detection quality.
- Energy-aware perception and security topics (adversarial patches, GPS spoofing effects on geo-referencing).

Appendix A. Files and folders

Path (in ~/drone_px4_isaac)	Purpose
start_app.sh, app.py	Launcher application
run_sim.sh, launch_custom_drone.py	Isaac Sim + Pegasus + PX4 simulation (command line)
assets/pegasus.usd, assets/x500/, assets/fpv/	Drone models
assets/baylands/, assets/car/, assets/person/	Scene and targets (USD + textures)
assets/src/	Original Gazebo Fuel downloads
assets/QGroundControl.AppImage	Ground station
models/best.pt	YOLOv8s VisDrone weights
venv/	Python 3.10 environment (torch, ultralytics, PyQt5, OpenCV)
tools/mesh_to_usd.py, add_colliders.py, build_drones.py	Asset and drone builders
mavlink_smoke_test.py, view_camera.py	Scripted take-off/landing; camera viewer and frame grabber
logs/app_test/	Test scripts, logs and frames used in this report
GUIDE.md	Detailed technical guide
docs/	This report (DOCX, PDF) and its generator
~/PX4-Autopilot/ROMFS/.../airframes/ 22001_isaac_x500, 22002_isaac_fpv	PX4 airframes added for Isaac

Appendix B. Commands

```
# the app
~/drone_px4_isaac/start_app.sh

# same scene from a terminal (X500, gimbal 35 degrees down)
cd ~/drone_px4_isaac
./run_sim.sh --usd assets/x500/x500.usd --px4-airframe isaac_x500 \
    --rotor-constant 8.54858e-6 --env baylands --objects --cam-tilt 35

# move the gimbal from any program
python3 -c "import socket,json; socket.socket(socket.AF_INET,socket.SOCK_DGRAM).sendto(
    json.dumps({'pitch':90,'yaw':0}).encode(), ('127.0.0.1',14600))"

# rebuild the drone models
python tools/build_drones.py x500 assets/x500/x500.usd
python tools/build_drones.py fpv  assets/fpv/fpv.usd

# leftover PX4 after a crash
pkill -x px4
```

Appendix C. Asset sources and licences

Baylands, Hatchback red (wheel texture from Hatchback) and Standing person come from Gazebo Fuel (fuel.gazebosim.org, OpenRobotics); their licences are stated on each model's Fuel page and in assets/src. The X500 meshes come from PX4-Autopilot (BSD-3). The Pegasus drone is the author's pegasus.usd. Isaac Sim, Pegasus (BSD-3), PX4 (BSD-3), QGroundControl (Apache-2.0/GPLv3) and Ultralytics (AGPL-3.0) are used under their respective licences.